Министерство науки и высшего образования Российской Федерации
Пензенский государственный университет
Кафедра «Вычислительная техника»

ОТЧЕТ

по лабораторной работе №9

по курсу «Логика и основы алгоритмизации в инженерных задачах»

на тему «Поиск расстояний в графе»

Выполнили:
студенты группы 25ВВВ3
Шибанов Ф.Д.
Флеонов М.В.

Пенза 2026

Цель работы и описание алгоритмов

Цель работы: найти кратчайшие расстояния в невзвешенном графе с помощью обходов в ширину и в глубину для матрицы и списков смежности, сравнить результаты и время выполнения. Выполнены основные и все задания со звёздочкой.

Представление графа

Вершины обозначаются числами от 1 до n. Матрица смежности содержит нули при отсутствии ребра. Для неориентированного графа она симметрична, диагональ нулевая. Внутри программы используются индексы от 0 до n − 1; при выводе номера увеличиваются на единицу.

Списки смежности построены как односвязные списки: узел хранит номер соседней вершины и ссылку на следующий узел. Соседи просматриваются по возрастанию номеров. Преобразование матрицы в списки выполняется один раз до запуска алгоритмов.

Генерация графа

Матрица генерируется функцией rand. В начале программы выполняется srand((unsigned)time(NULL)), чтобы начальное состояние генератора зависело от текущего времени. Для каждой пары разных вершин вероятность наличия ребра задаётся параметром p в процентах. В неориентированном графе значения записываются симметрично. Пользователю не требуется задавать начальное состояние генератора.

Принцип обхода в ширину

Начальная вершина помечается и помещается в FIFO-очередь. Затем вершина извлекается из начала очереди, а её ещё не посещённые соседи помечаются и добавляются в конец. Пометка выполняется при добавлении, поэтому вершина не попадает в очередь повторно. После завершения компоненты поиск возобновляется из первой непосещённой вершины, если нужен полный обход графа.

Среда выполнения

Программа написана на C++. Для сборки достаточно команды g++ main.cpp -o lab9. Запуск и измерения выполнены в Linux x86_64. Порядок графа ограничен 2000 вершинами для ограничения объёма матрицы.

Задание 1 Поиск расстояний обходом в ширину

Пункт 1: сгенерирована и выведена матрица неориентированного графа. Пункт 2: используется std::queue<int>; исходная вершина получает расстояние 0, остальные — значение −1. При обнаружении ещё не посещённого соседа v присваивается dist[v] = dist[u] + 1. Пункт 3 со звёздочкой: тот же алгоритм реализован для связных списков смежности.

В отличие от полного обхода в работе №8, поиск расстояний запускается только из выбранной вершины. Изолированные от неё компоненты не получают фиктивных расстояний: для них сохраняется −1. В псевдокоде методички обозначение непосещённой вершины следует понимать как −1, поскольку значение 1 уже является допустимым расстоянием.

```
$ ./lab9 --n 6 --p 40
ЛР9. Расстояния. n=6, p=40%, start=1
Матрица смежности:
       1   2   3   4   5   6
  1    0   0   0   1   0   1
  2    0   0   0   1   1   1
  3    0   0   0   1   1   1
  4    1   1   1   0   1   0
  5    0   1   1   1   0   0
  6    1   1   1   0   0   0
Списки смежности:
1: 4 6 -> NULL
2: 4 5 6 -> NULL
3: 4 5 6 -> NULL
4: 1 2 3 5 -> NULL
5: 2 3 4 -> NULL
6: 1 2 3 -> NULL
BFS матрица: 0 2 2 1 2 1
BFS списки: 0 2 2 1 2 1
DFS матрица с релаксацией: 0 2 2 1 2 1
DFS списки с релаксацией: 0 2 2 1 2 1
-1 означает недостижимую вершину.
```

Рисунок 1 — Совпадение расстояний BFS и DFS для двух представлений

Для демонстрационного графа из вершины 1 получен вектор 0 2 2 1 2 1. Первое значение равно 0; единицами обозначены непосредственные соседи, а −1 соответствует недостижимым вершинам. Все четыре строки результатов совпадают.

Задание 2 Поиск расстояний обходом в глубину

Пункты 1 и 2 со звёздочкой DFS по матрице и спискам

Обычный DFS первого посещения находит некоторый путь, который может быть длиннее кратчайшего. Поэтому применяется повторная релаксация: при обнаружении вершины впервые или при условии dist[v] > dist[u] + 1 её расстояние уменьшается и из неё вновь рекурсивно просматриваются соседи. В матрице перебирается строка, в списках — только узлы текущего списка.

Корректность обеспечивается тем, что каждое конечное расстояние соответствует существующему пути. Улучшение возможно только на меньшее неотрицательное целое число. После завершения для каждого ребра выполнено dist[v] ≤ dist[u] + 1; следовательно, расстояния кратчайшие. На циклах рекурсия не бесконечна, поскольку равное или большее расстояние не вызывает повторного входа.

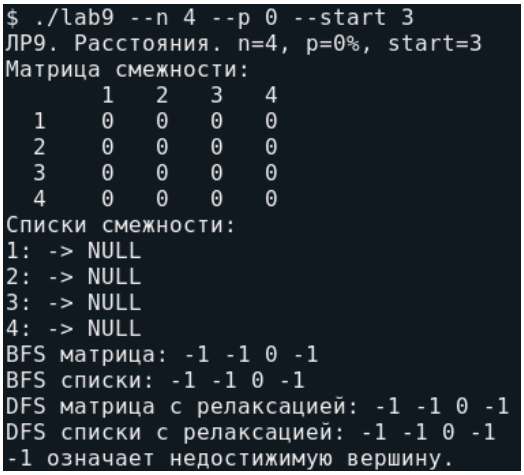


Рисунок 2 — Недостижимые вершины сохраняют расстояние −1

При n = 4, p = 0 и старте из вершины 3 все четыре реализации возвращают −1 −1 0 −1. Это подтверждает, что расстояния не переносятся между разными компонентами.

Оценка времени работы

Замеры: среднее 10 поисков; p=10%; Время CPU, мс.

Для каждого порядка генерируется один граф, который используется всеми сравниваемыми реализациями. Генерация, построение списков и вывод исключены из измеряемого участка. Создание рабочей очереди и рабочих массивов внутри алгоритма включено. Повторные запуски усредняются; значения зависят от нагрузки компьютера и не являются универсальными константами.

n	**BFS матрица, мс**	**BFS списки, мс**	**DFS матрица, мс**	**DFS списки, мс**
50	0.022000	0.008500	0.326300	0.051500
100	0.079800	0.021400	3.606700	0.455700
200	0.305600	0.064700	28.109400	2.804600
400	0.828000	0.103000	211.003400	21.490300

```
$ ./lab9 --benchmark
Замеры: среднее 10 поисков; p=10%; Время CPU, мс.
n; bfs_matrix_ms; bfs_list_ms; dfs_matrix_ms; dfs_list_ms
50; 0.022000; 0.008500; 0.326300; 0.051500
100; 0.079800; 0.021400; 3.606700; 0.455700
200; 0.305600; 0.064700; 28.109400; 2.804600
400; 0.828000; 0.103000; 211.003400; 21.490300
Контрольная сумма: 360
```

Рисунок 3 — Результаты измерения времени

BFS посещает достижимую вершину один раз: матричная реализация имеет сложность $O(n^2)$, списочная — $O(n + m)$. DFS с релаксацией повторно раскрывает вершины при сокращении пути и на исследованных графах работает существенно дольше. Грубые верхние оценки для этой реализации: $O(n^3)$ по матрице и $O(n(n + m))$ по спискам; число улучшений расстояния каждой вершины ограничено n.

Базовые проверки и вывод

Базовые проверки

Для графа из четырёх изолированных вершин при старте из вершины 3 все реализации дают расстояния −1 −1 0 −1. Для графа из одной вершины расстояние до неё равно 0. В полном графе из четырёх вершин при старте из вершины 1 расстояния равны 0 1 1 1.

Файлы и запуск

main.cpp содержит алгоритмы и точку входа; graph.h — представление графа, генерацию, ввод, вывод и разбор параметров. В папке также находится готовый исполняемый файл. Для сборки используется одна команда компиляции.

```
g++ main.cpp -o lab9
./lab9 --n 6 --p 40
```

Вывод

Реализован поиск кратчайших расстояний на основе BFS и DFS с повторной релаксацией для матриц и связных списков. Все варианты дают одинаковые расстояния, но DFS выполняет дополнительные раскрытия вершин. Для невзвешенного графа BFS оказался более подходящим способом поиска расстояний.